

MODULE 14

Deep Learning

Topics Covered

- Lecture 1 — Introduction to Deep Learning
- Lecture 2 — Working of Deep Learning & ML vs DL
- Lecture 3 — Perceptron in Deep Learning
- Lecture 4 — Multi-Layered Perceptron (MLP)

Lecture 1: Introduction to Deep Learning

Deep Learning (DL) is a subset of Machine Learning that uses artificial neural networks with many layers to learn representations of data. Instead of hand-crafting features, DL models learn features automatically — directly from raw inputs like pixels, audio waveforms, or text tokens.

□ The hierarchy

Artificial Intelligence $\supset$ Machine Learning $\supset$ Deep Learning.

Deep Learning = ML with neural networks that have more than one hidden layer.

The 'deep' refers to the depth (number of layers), not complexity of thought.

Why Deep Learning Now?

Driver	Details
Big Data	DL needs huge amounts of data to outperform classical ML — the internet provides it
GPUs	Parallel matrix computation on GPUs makes training 100× faster than CPUs
Better algorithms	ReLU activation, batch norm, dropout, Adam optimiser — all post-2010 innovations
Frameworks	TensorFlow (Google, 2015) and PyTorch (Meta, 2016) made DL accessible
Pre-trained models	Transfer learning — reuse a model trained on millions of images for your task

Where Deep Learning Excels

Domain	DL Application
Computer Vision	Image classification, object detection, face recognition, medical imaging
NLP	Sentiment analysis, translation, chatbots, summarisation (BERT, GPT)
Speech	Speech recognition (Whisper), text-to-speech (WaveNet)
Generative AI	Image generation (DALL·E, Stable Diffusion), LLMs (GPT-4, Claude)
Healthcare	Disease detection from X-rays/MRI, drug discovery
Games & Robotics	AlphaGo, self-driving cars, robotic control

DL vs Classical ML — When to Use Each

Situation	Choose
Small dataset (< 10K rows)	Classical ML (Random Forest, SVM, XGBoost)
Tabular / structured data	Classical ML — usually faster and more interpretable
Images, audio, video, raw text	Deep Learning — automatic feature extraction shines

Situation	Choose
Large dataset (millions of samples)	Deep Learning — scales better
Need explainability / fast training	Classical ML
State-of-the-art accuracy is critical	Deep Learning

Lecture 2: Working of Deep Learning & ML vs DL

How Deep Learning Works — Big Picture

A deep neural network takes an input, passes it through many layers of mathematical transformations, and produces an output. Each layer learns progressively more abstract representations of the data.

Layer	What It Learns (Image Example)
Layer 1 (early)	Simple edges and colour gradients
Layer 2	Shapes — circles, lines, corners
Layer 3	Parts — eyes, wheels, doors
Layer 4 (deep)	Objects — faces, cars, dogs
Output layer	Final prediction — 'this is a cat' (92% confidence)

□ Key idea

You never manually tell the network 'look for edges in layer 1'.
The network discovers these representations by itself during training — through gradient descent.

The Training Process

Step	What Happens
1. Forward pass	Input data flows through the network layer by layer → produces prediction $\hat{y}$
2. Loss calculation	Compare prediction $\hat{y}$ to true label y → compute loss (e.g. cross-entropy or MSE)
3. Backward pass	Backpropagation: compute gradient of loss w.r.t. every weight using chain rule
4. Weight update	Optimiser (e.g. Adam) adjusts weights in the direction that reduces loss
5. Repeat	Repeat steps 1–4 for all batches across many epochs until loss converges

ML vs Deep Learning — Detailed Comparison

Aspect	Classical ML vs Deep Learning
Feature engineering	Manual — domain expert extracts features vs Automatic — model learns features
Data requirement	Works well on small data (100s–10Ks) vs Needs large data (100Ks–millions)

Aspect	Classical ML vs Deep Learning
Compute	CPU is sufficient vs GPU/TPU practically required
Training time	Seconds to minutes vs Hours to days (sometimes weeks)
Interpretability	High — e.g. Decision Tree can be visualised vs Low — 'black box'
Architecture	Fixed by algorithm choice vs Highly customisable (layers, neurons, connections)
Transfer learning	Limited vs Core strength — fine-tune pretrained models
Examples	Random Forest, SVM, XGBoost vs CNN, RNN, LSTM, Transformer

Key Deep Learning Concepts

Concept	Definition
Epoch	One complete pass through the entire training dataset
Batch size	Number of samples processed before weights are updated (e.g. 32, 64, 128)
Iteration	One weight update = processing one batch. Iterations per epoch = dataset_size / batch_size
Learning rate	How large a step the optimiser takes. Too high → diverge; too low → very slow
Loss function	Measures how wrong the prediction is. Goal: minimise this over training
Optimiser	Algorithm that updates weights. SGD, Adam (most popular), RMSprop, AdaGrad

Lecture 3: Perceptron in Deep Learning

The Perceptron is the simplest neural network building block — a single artificial neuron. Inspired by biological neurons, it takes multiple inputs, computes a weighted sum, applies an activation function, and produces an output.

□ Biological analogy

Dendrites = inputs ($x_1, x_2, \dots, x_n$)

Synapse strength = weights ($w_1, w_2, \dots, w_n$)

Cell body = weighted sum + activation function

Axon = output signal

Perceptron Mathematics

Component	Formula Role
Weighted sum (z)	$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = W \cdot X + b$
Bias (b)	Shifts the activation threshold — allows the model to fit data not passing through origin
Activation ($\hat{y}$)	$\hat{y} = \text{activation}(z)$ — introduces non-linearity
Step function	Output 1 if $z \geq 0$, else 0 — original Perceptron activation (binary only)

Component	Formula Role
<pre>import numpy as np # Single perceptron - manual implementation class Perceptron: def __init__(self, n_inputs, lr=0.1): self.w = np.zeros(n_inputs) # weights self.b = 0 # bias self.lr = lr def predict(self, X): z = np.dot(X, self.w) + self.b return np.where(z >= 0, 1, 0) # step activation def fit(self, X, y, epochs=10): for _ in range(epochs): for xi, yi in zip(X, y): y_hat = self.predict(xi) error = yi - y_hat self.w += self.lr * error * xi # weight update self.b += self.lr * error # bias update # Train on AND gate X = np.array([[0,0],[0,1],[1,0],[1,1]]) y = np.array([0, 0, 0, 1]) # AND output p = Perceptron(n_inputs=2) p.fit(X, y, epochs=20) print(p.predict(X)) # [0, 0, 0, 1] ✓</pre>	

Activation Functions

The activation function determines whether and how strongly a neuron 'fires'. Without activation functions, a neural network is just a linear model — no matter how many layers.

Activation	Formula Range Use Case
Step	1 if $z \geq 0$ else 0 {0,1} Original perceptron only — not differentiable
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$ (0,1) Output layer for binary classification
Tanh	$\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$ (-1,1) Hidden layers — zero-centred
ReLU	$\max(0, z)$ $[0, \infty)$ Default for hidden layers — fast, avoids vanishing gradient
Leaky ReLU	z if $z > 0$ else $0.01z$ $(-\infty, \infty)$ Fixes 'dying ReLU' problem
Softmax	$e^{z^i} / \sum e^{z^j}$ (0,1) sum=1 Output layer for multi-class classification
<pre>import numpy as np def sigmoid(z): return 1 / (1 + np.exp(-z)) def tanh(z): return np.tanh(z) def relu(z): return np.maximum(0, z) def leaky_relu(z): return np.where(z > 0, z, 0.01 * z) def softmax(z): e = np.exp(z - np.max(z)); return e / e.sum() z = np.array([-2, -1, 0, 1, 2]) print('ReLU: ', relu(z)) # [0, 0, 0, 1, 2] print('Sigmoid:', sigmoid(z).round(3)) # [0.119, 0.269, 0.5, 0.731, 0.881]</pre>	

Limitation of Single Perceptron

A single perceptron can only learn linearly separable problems. It famously cannot learn the XOR function — a problem that triggered the 'AI winter' of the 1970s.

Problem	Single Perceptron
AND gate	✓ Learnable — linearly separable
OR gate	✓ Learnable — linearly separable
NOT gate	✓ Learnable — linearly separable
XOR gate	✗ Not learnable — NOT linearly separable
□ Solution Stack multiple perceptrons in layers → Multi-Layer Perceptron (MLP). See Lecture 4.	

Lecture 4: Multi-Layered Perceptron (MLP)

A Multi-Layer Perceptron (MLP) is a fully connected feedforward neural network with one or more hidden layers between the input and output. Adding hidden layers lets the network learn non-linear decision boundaries — solving problems like XOR that a single perceptron cannot.

MLP Architecture

Layer Type	Role Neurons
Input layer	One neuron per feature — no computation, just passes data in
Hidden layer(s)	Where learning happens. Each neuron = weighted sum + activation. Can have many hidden layers.
Output layer	1 neuron (regression/binary), N neurons (multi-class with softmax)
# Example: 3-layer MLP for binary classification # Input: 4 features → Hidden: 8 neurons → Output: 1 neuron (sigmoid) # Layer 1 (Hidden): $z^1 = W^1 \cdot X + b^1$, $a^1 = \text{ReLU}(z^1)$ # Layer 2 (Output): $z^2 = W^2 \cdot a^1 + b^2$, $\hat{y} = \text{sigmoid}(z^2)$ # Number of parameters: # W^1 : $(4 \times 8) = 32$ weights + 8 biases = 40 # W^2 : $(8 \times 1) = 8$ weights + 1 bias = 9 # Total: 49 learnable parameters	

Backpropagation — How MLP Learns

Backpropagation uses the chain rule of calculus to compute the gradient of the loss with respect to every weight in the network, then updates them to reduce loss.

Step	Description
Forward pass	Compute predictions layer by layer: $X \rightarrow a^1 \rightarrow a^2 \rightarrow \hat{y}$
Compute loss	$L = \text{loss}(y, \hat{y})$ e.g. binary cross-entropy or MSE
Output gradient	$\partial L / \partial \hat{y}$ — how much does loss change with output?
Chain rule back	$\partial L / \partial W^2 \rightarrow \partial L / \partial a^1 \rightarrow \partial L / \partial W^1$ — propagate gradients layer by layer
Weight update	$W = W - \alpha \times \partial L / \partial W$ where α = learning rate

Building an MLP with Keras

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

# XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0, 1, 1, 0], dtype=float) # XOR output

# Build model
model = keras.Sequential([
    layers.Dense(4, activation='relu', input_shape=(2,)), # hidden
    layers.Dense(1, activation='sigmoid') # output
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
# Total params: 13 (4×2+4 + 4×1+1)

# Train
history = model.fit(X, y, epochs=1000, verbose=0)

# Predict
print(model.predict(X).round())
# [[0.], [1.], [1.], [0.]] ✓ - XOR solved!
```

Building a Deeper MLP — Classification Example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Data
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(64, activation='relu', input_shape=(4,)),
    Dropout(0.3), # randomly drop 30% neurons - prevents overfitting
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(3, activation='softmax') # 3 classes
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```

history = model.fit(X_train, y_train,
                    epochs=100,
                    batch_size=16,
                    validation_split=0.2,
                    verbose=1)

loss, acc = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {acc:.4f}')

```

Key Hyperparameters in MLP

Hyperparameter	Effect Typical Values
Number of layers	Deeper = more complex patterns. Start with 2–3 hidden layers.
Neurons per layer	More neurons = larger capacity. Powers of 2: 32, 64, 128, 256.
Activation function	ReLU for hidden layers; sigmoid/softmax for output.
Learning rate	Controls step size. Too high → diverge; too low → slow. Try 0.001 (Adam default).
Batch size	32 or 64 most common. Larger = stable gradient; smaller = noisy but regularising.
Epochs	Train until validation loss stops improving. Use EarlyStopping callback.
Dropout rate	0.2–0.5 in hidden layers. Randomly disables neurons to reduce overfitting.

Callbacks — Smarter Training

```

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

callbacks = [
    EarlyStopping(
        monitor='val_loss',    # watch validation loss
        patience=10,           # stop if no improvement for 10 epochs
        restore_best_weights=True
    ),
    ModelCheckpoint(
        'best_model.h5',
        save_best_only=True    # save only the best epoch's weights
    )
]

model.fit(X_train, y_train, epochs=500,
          validation_split=0.2,
          callbacks=callbacks)  # training auto-stops at best point

```

Loss Functions — Choosing the Right One

Task	Loss Function Keras Name
Binary classification (2 classes)	Binary Cross-Entropy → 'binary_crossentropy'
Multi-class (integer labels)	Sparse Categorical Cross-Entropy → 'sparse_categorical_crossentropy'
Multi-class (one-hot labels)	Categorical Cross-Entropy → 'categorical_crossentropy'

Task	Loss Function Keras Name
Regression	Mean Squared Error → 'mse' or Mean Absolute Error → 'mae'

End of Module 14 Notes — Generative AI Course